AIMeetAbuja · HANDS-ON WORKSHOP · 2026

AI-Native Software Engineering

From generating code to engineering reliable software

Joshua Okechukwu · Co-founder, 1Talent Africa

90-minute hands-on workshop · Bring your laptop and your AI coding agent

AIMeetAbuja

1Talent Africa · 2026

IMPACT HUB ABUJA

2nd Floor, 21 Blantyre Cres,
off Adetokunbo Ademola Crescent,
Wuse II

Generating code is not the same as engineering software

AI can accelerate implementation. Engineering judgment determines whether the result is reliable.

APPROACH A

One-shot generation

1. Describe an idea.
2. Generate an application.
3. See a working interface.
4. Assume it is finished.

The demo is the deliverable.

APPROACH B

AI-native engineering

1. Establish context.
2. Investigate the codebase.
3. Define the problem.
4. Plan and implement a change.
5. Verify the result.

The evidence is the deliverable.

A working demo is not proof of production readiness.

Generate → Investigate → Engineer

Three moves. The third one is where most of the work lives.

01

Generate

Create a starting point quickly — a plausible application, not a finished one.

02

Investigate

Understand the code, the data model, existing behaviour, and the failure modes.

03

Engineer

Specify the change, plan the smallest safe implementation, then verify it.

↺ **Iteration is the default.** Investigation can revise the plan. Verification can send you back to implementation.

Do not confuse the speed of generating an application with the quality of the engineering process.

Our workshop project: Gather

A small event app — and a codebase with real behaviour to investigate.

THE CODEBASE

01 Browse events

02 View event details

03 Register with name + email

04 Manage events as an organiser

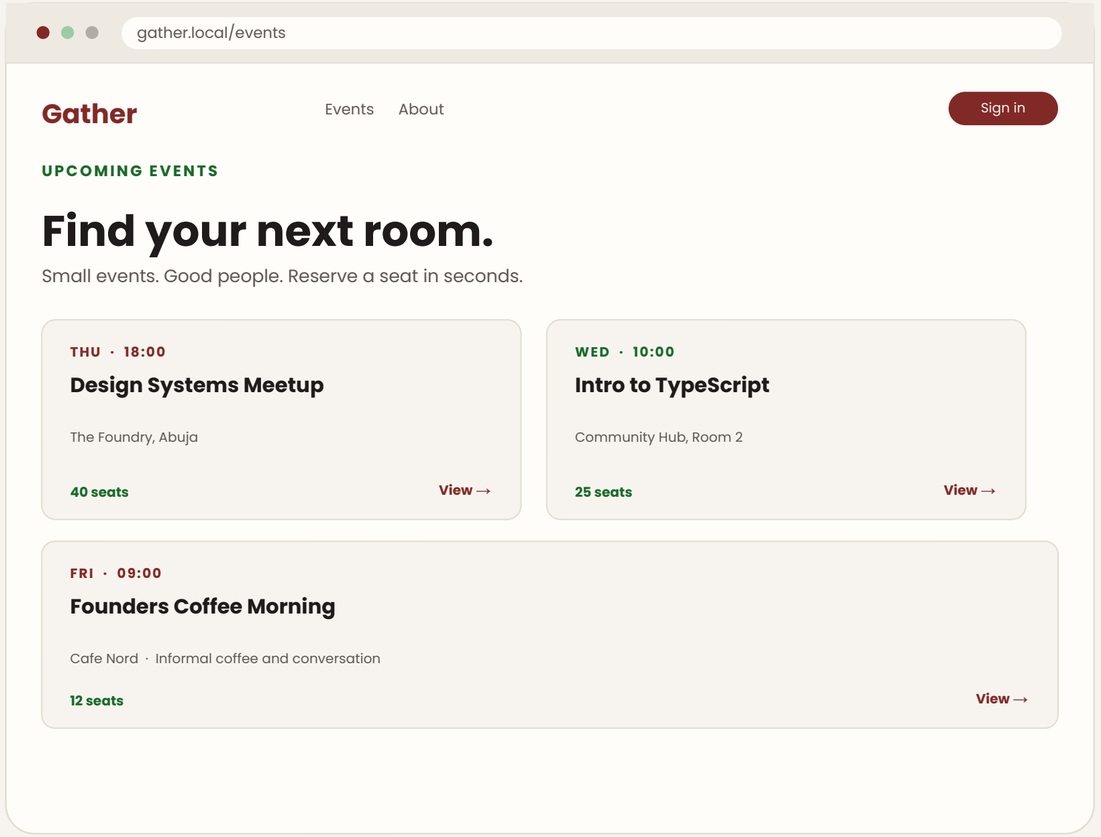
THE STACK

Frontend	React · Vite · TypeScript
API	Hono
Database	SQLite
Styling	Tailwind CSS · shadcn/ui

REPOSITORY

https://github.com/Parallax18/gather

The app is the starting point for investigation, not the proof.



Interface illustration — the workshop starts from the real repository.

First, build the obvious version

Prompt broadly. Then investigate everything the prompt did not decide.

PROMPT 01 · THE OBVIOUS VERSION

"Build a simple event management application called Gather. Users should be able to browse events, view event details, and register for events. Organisers should be able to create and manage events. Use the existing project stack and design a clean, responsive interface. Keep the implementation simple and easy to use."

Broad by design. The details it leaves out become the investigation.

≈ 70 words

What it leaves open

Business rules
Who may register, how many seats, when registration closes.

Failure scenarios
What happens when an event is full, past, or unpublished.

Access control
Can one organiser change another organiser's event?

Concurrency
Two people, one seat, the same instant.

The generated result is a starting point for investigation — not evidence that the application is correct.

What could go wrong?

Five questions to investigate — not five confirmed bugs.

SCENARIO 01

The last seat

Two users register at almost the same time. Only one seat remains. Can both succeed?

SCENARIO 02

Duplicate registrations

Can the same email address register for the same event more than once?

SCENARIO 03

Unpublished events

An organiser unpublishes an event while someone is registering. What should happen — and what does?

SCENARIO 04

Access control

Can one organiser view or modify another organiser's event?

SCENARIO 05

Retried requests

A request times out and the client retries. Could the user end up registered twice?

ASK YOUR CODING AGENT TO

- Trace the request and data flow
- Identify the code and database constraints
- Explain what the code does today
- Give evidence for each conclusion
- Suggest focused tests

Separate confirmed behaviour from potential risk. A plausible explanation is not evidence.

A disciplined way to work with a coding agent

Five steps. One loop. The evidence decides when it ends.

1

Understand

Inspect the files, data relationships, existing behaviour, and constraints.

2

Specify

Define expected behaviour — including edge cases and failure conditions.

3

Plan

Find the smallest safe change and the tests it needs.

4

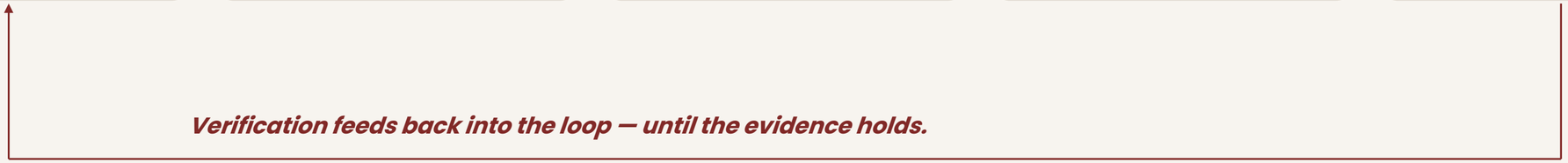
Implement

Let the agent make a targeted change.

5

Verify

Review the diff, run the tests, and confirm the specification.

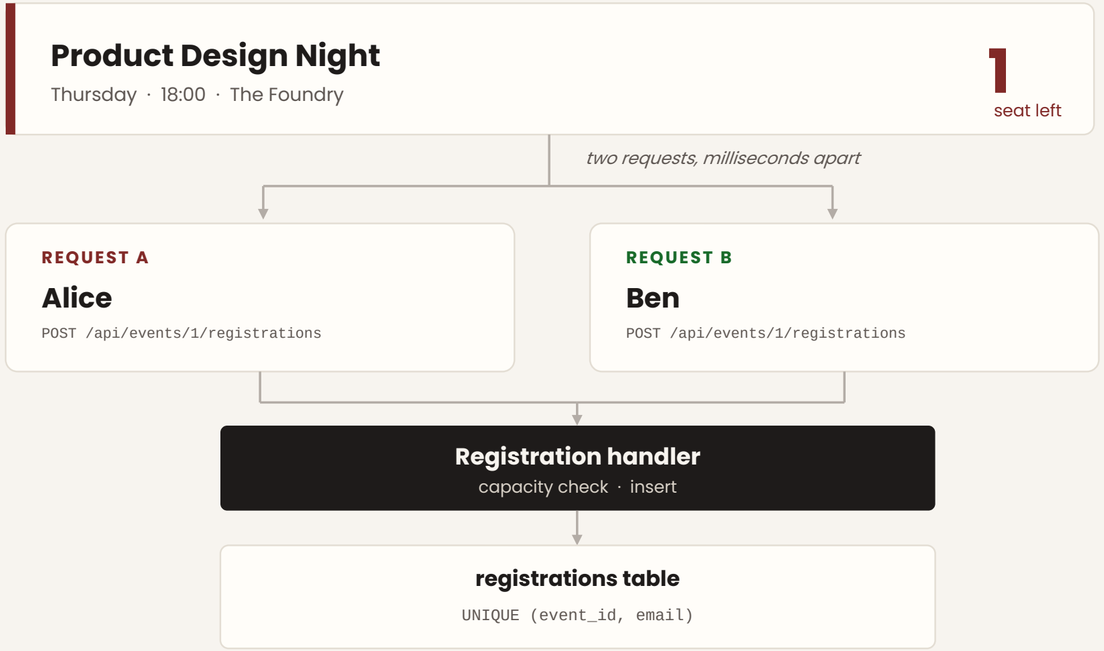


Inspect the evidence. Do not just accept the conclusion.

One seat. Two users. What happens?

A scenario to investigate — not a diagnosis of the code.

THE SCENARIO



The diagram shows the path to inspect — not the result.

DESIRED BEHAVIOUR

- ✓ At most one registration succeeds.
- ✓ The other request receives a clear, appropriate failure response.
- ✓ The event never exceeds its capacity.
- ✓ The database remains consistent.

INVESTIGATE

- Registration handler
- Database schema and constraints
- Transaction boundaries
- Capacity checks
- Concurrent execution tests

What would you need to inspect or test to prove this works?

How do we know the change is safe?

Review the work, not just the demo.

01 · BEHAVIOUR

- ☐ **Requirements met**
  The implementation does what the brief asked.
- ☐ **Edge cases considered**
  Full, duplicate, past, unpublished, retried.
- ☐ **Existing behaviour intact**
  Nothing unrelated regressed.

02 · DATA & ACCESS

- ☐ **Constraints and transactions**
  The database enforces what the code assumes.
- ☐ **Permissions on the server**
  Not just hidden in the interface.
- ☐ **Duplicates and retries**
  Safe under repeated or concurrent requests.

03 · EVIDENCE

- ☐ **Tests added and run**
  Relevant checks, not just a successful build.
- ☐ **The diff reviewed**
  Read the change; do not skim the summary.
- ☐ **Claims supported**
  Every conclusion has evidence behind it.

“The agent says it works” is not a verification strategy.

Agents are useful. Their conclusions still need evidence.

FOUR PRINCIPLES

Build with AI.
Think like an engineer.

01
Context before code

Help the agent understand the project before asking for changes.

02
Investigate before assuming

Treat hypotheses as questions until evidence confirms them.

03
Specify before implementing

Define expected behaviour and failure conditions.

04
Verify before trusting

Inspect the diff, run the tests, validate the result.

AI accelerates implementation. Engineering judgment decides what ships.

IMPACT HUB Abuja | 1talentafrica

THE POINT

The goal is not to generate more code. It is to build better software with AI.

Thank you

Questions & discussion

Joshua Okechukwu · Co-founder, 1Talent Africa

AIMeetAbuja · 2026

IMPACT HUB ABUJA
2nd Floor, 21 Blantyre Cres,
off Adetokunbo Ademola Crescent, Wuse II